Trainee Task — Public Libraries Mini-Pipeline (Airflow + Spark)

Nov 5, 2026 · @ali obaid

Goal

Build a small **Bronze → Silver → Gold** pipeline with **PySpark + Delta Lake**, orchestrated by **one Airflow DAG**, that answers one business question: *how did US public library usage recover from COVID between FY2020 and FY2022, state by state?*

This is a one-week onboarding task. It mirrors how the Data Fabric works (medallion layers, Delta tables, Airflow-triggered Spark jobs) on a small, public dataset, so you can learn the patterns without touching client data.

What you will learn: reading messy CSVs in Spark, handling schema drift across years, special-value codes, Delta writes and partitioning, idempotent jobs, and wiring Spark jobs into an Airflow DAG with retries and data checks.

Dataset

IMLS Public Libraries Survey (PLS), **Administrative Entity (AE)** files, imputed versions. One row = one library system for one fiscal year. All files are in the data/ folder of the task package; the FY21 and FY22 README files are in docs/.

File	Fiscal year	Rows	Columns	Encoding
PLS_FY20_AE_pud20i.csv	2020	9,245	179	cp1252
PLS_FY21_AE_pud21i.csv	2021	9,215	195	utf-8
PLS_FY22_AE_pud22i.csv	2022	9,248	192	cp1252

The data is deliberately realistic. These are the traps you must handle (read the README files first):

- **Different encodings per year.** Reading cp1252 as UTF-8 corrupts library names.
- **Schema drift.** Column count changes every year (e.g. FY21 adds `TOTPHYS`, `ADULTPRO`; FY22 adds `ODFINE`, `LSAGEOID` and drops `INCITSST`).
- **Special negative codes, not real numbers:** `-1` missing, `-3` temporarily closed, `-4` not applicable, `-9` suppressed for confidentiality. Summing them as numbers gives wrong totals.

- **Temporarily closed libraries** have `STATSTRU = 23` (4 in FY20, 8 in FY21, 10 in FY22) and all their metrics set to `-3`.
- **`-3` also appears outside code 23.** Some FY20 libraries with a different `STATSTRU` value carry `-3` too. Find them and document how Silver treats them.
- **Flag columns** (`F_*`) sit beside each metric and describe how it was imputed. Keep them in Bronze, not needed in Gold.

Key columns: `FSCSKEY` (library ID), `STABR` (state), `LIBNAME`, `POPU_LSA` (population served), `VISITS`, `TOTCIR` (total circulation), `ELMATCIR` (e-circulation), `REGBOR` (registered users), `TOTOPEXP` (operating expenditure, USD), `TOTSTAFF` (FTE), `HRS_OPEN`, `TOTPRO` / `TOTATTEN` (programs / attendance), `STATSTRU`.

Requirements

Environment. Start locally: PySpark (local mode) + `delta-spark` + Airflow (standalone or the official docker-compose). Store Delta tables on a local folder or a local MinIO container via S3A. Moving it to the K3s cluster is a stretch goal, not required.

1. Bronze job — `bronze_ingest.py`

1. Read each yearly CSV with its correct encoding, all columns as **string** (no type inference).
2. Add metadata columns: `fiscal_year`, `source_file`, `ingestion_ts`.
3. Write to one Delta table `bronze.pls_ae`, partitioned by `fiscal_year`, with schema merge enabled so new columns per year don't break the write.
4. Must be **idempotent**: re-running a year overwrites that year's partition only (no duplicates).

2. Silver job — `silver_clean.py`

1. Select only the key columns listed in the Dataset section (same set for all years).
2. Cast to proper types: counts → `long`, `TOTSTAFF` → `double`, `fiscal_year` → `int`.
3. Convert the special codes `-1`, `-3`, `-4`, `-9` to `null`, and keep the original reason in one column `missing_reason` for the record (or per metric — explain your choice).
4. Exclude temporarily closed libraries (`STATSTRU = 23`) and log how many were removed per year.
5. Deduplicate on (`FSCSKEY`, `fiscal_year`).
6. Write `silver.library_year`, partitioned by `fiscal_year`.

3. Gold job — `gold_state_kpis.py`

One row per (`STABR`, `fiscal_year`) in `gold.state_year_kpis` with:

- `libraries` (count), `population_served`, `total_visits`, `total_circulation`
- `visits_per_capita` = total_visits / population_served
- `circulation_per_capita`, `e_circ_share` = ELMATCIR / TOTCIR
- `opex_per_capita` (USD)
- `visits_yoy_pct`: change vs. the previous fiscal year of the same state (window function)

4. Airflow DAG — `pls_pipeline_dag.py`

1. Tasks: `check_source_files` → `bronze_ingest` → `silver_clean` → `quality_check` → `gold_state_kpis`.
2. Pass `fiscal_year` as a DAG param so one year can be re-run alone; default = all three.
3. `quality_check` fails the DAG if any rule below is broken (plain PySpark asserts are fine; Great Expectations is a stretch goal).
4. Retries = 2, a clear `doc_md`, no hard-coded paths (use Airflow Variables or a config file).
5. Spark jobs triggered with `SparkSubmitOperator` locally (or `SparkKubernetesOperator` on K3s).

Acceptance criteria and deliverables

The task is done when every box below is ticked and demoed in a 20-minute review.

Data checks (the `quality_check` task enforces these):

- [ ] Bronze row counts match the source files exactly: 9,245 / 9,215 / 9,248 (27,708 total).
- [ ] Silver has no row with `STATSTRU = 23`; FY20 drops 4, FY21 drops 8 and FY22 drops 10 closed libraries.
- [ ] No negative value remains in any Silver metric column.
- [ ] `FSCSKEY` + `fiscal_year` is unique in Silver; `FSCSKEY` and `STABR` are never null.
- [ ] Gold has one row per state per year, and `visits_yoy_pct` is null only in each state's first year in the data (careful: one state appears only in FY2022).
- [ ] `LIBNAME` values with accents (e.g. PR library names in FY2022) read correctly — proves encoding is right.

Engineering checks:

- [ ] Running the DAG twice gives the same row counts (idempotent).

- [ ] Re-running with `fiscal_year=2021` touches only the 2021 partitions.
- [ ] Delta time travel works: show `bronze.pls_ae` at version 0.

Deliverables:

- [ ] Git repo: `dags/`, `jobs/`, `config/`, `README.md` (how to run, design choices, known limits).
- [ ] Screenshot of a green DAG run in the Airflow UI.
- [ ] A short findings note: the top 5 and bottom 5 states by visits recovery FY20 → FY22, plus one chart.
- [ ] Small PRs, one per job, reviewed before merge.

Plan, hints and evaluation

Day	Focus	End-of-day output
1	Set up Spark + Delta + Airflow locally; read the READMEs; explore the CSVs in a notebook	Environment runs; list of data issues found
2	Bronze job	`bronze.pls_ae` with correct counts, idempotent
3	Silver job	`silver.library_year` passing all data checks
4	Gold job + Airflow DAG	Green end-to-end DAG run
5	Quality task, README, findings note	Demo-ready repo

Hints:

- Spark reads CSV encoding with `.option("encoding", ...)`; check one accented `LIBNAME` before going further.
- For schema drift, look at `mergeSchema` on Delta writes. For partition-only overwrites, look at `replaceWhere`.
- Handle the special codes once, in a reusable function, not column by column.
- Use `Window.partitionBy("STABR").orderBy("fiscal_year")` with `lag()` for year-over-year.
- Commit early, push daily, and ask when blocked for more than an hour.

Stretch goals (only after the core is done):

- Validate with Great Expectations instead of plain asserts.

- Run the jobs on the K3s dev cluster with `SparkKubernetesOperator`, reading from MinIO.
- Cross-check your Gold FY2020 totals against `PLS_FY20_State_pud20i.csv` and explain any gap.
- Join the Outlet files to add branch counts per state.

How it will be evaluated: correctness of the numbers (40%), code quality and structure (25%), Airflow design — params, retries, idempotency (20%), README and findings note (15%).